RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

FASHION ITEM IMAGE CLASSIFICATION: BASELINE NEURAL NETWORK VS CNN

Submitted by

LATHIKA RAJMOHAN 241501098
NAKSHATH N 241501126

AI23531 DEEP LEARNING

Department of Artificial Intelligence and Machine Learning

Rajalakshmi Engineering College,

Rajalakshmi Nagar, Thandalam

Tamil Nadu – 602 105

OCTOBER 2026

RAJALAKSHMI ENGINEERING COLLEGE

BONAFIDE CERTIFICATE

NAME	Lathika Rajmohan	241501098	
	Nakshath N	241501126	
ACADEMIC YEAR	2026 – 2027	**SEMESTER**	V
BRANCH	Artificial Intelligence and Machine Learning		

UNIVERSITY REGISTER NO. **241501098**
241501126

Certified that this is the bonafide record of work done by the above student in the Mini Project titled **Fashion Item Image Classification: Baseline Neural Network vs CNN** in the subject **AI23531 Deep Learning** during the year **2026 - 2027**.

Signature of Faculty – in – Charge

Submitted for the Practical Examination held on _____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ABSTRACT

Image classification is an important application of deep learning and computer vision. Fashion item classification involves identifying clothing and accessories from digital images. This project, Fashion Item Image Classification: Baseline Neural Network vs CNN, implements and compares two deep learning models using the Fashion-MNIST dataset.

The dataset contains 70,000 grayscale images belonging to ten fashion categories. The images are resized to a standard resolution of 28 × 28 pixels and normalized before being supplied to the models. A baseline Multilayer Perceptron (MLP) is developed using a flattening layer and fully connected layers. A Convolutional Neural Network (CNN) is implemented using convolutional layers, max-pooling layers, a dense layer, and dropout regularization.

Both models are trained using TensorFlow and Keras for ten epochs. Their performance is evaluated using accuracy, precision, recall, and F1-score. Training and validation graphs, confusion matrices, and model comparison charts are generated to analyze their performance. The trained models are saved in Keras format for future use.

This project demonstrates the application of deep learning to image classification and provides a comparative analysis of a conventional neural network and a CNN for recognizing fashion items.

Keywords:

Deep Learning, Fashion-MNIST, Image Classification, Multilayer Perceptron, Convolutional Neural Network, TensorFlow, Keras, Computer Vision.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	**III**
1	**INTRODUCTION**	**1**
	1.1 INTRODUCTION	1
	1.2 OBJECTIVES	1
	1.3 SCOPE OF THE PROJECT	2
2	**LITERATURE REVIEW**	**3**
	2.1 DEEP LEARNING FOR IMAGE CLASSIFICATION	3
	2.2 MULTILAYER PERCEPTRON FOR CLASSIFICATION	3
	2.3 CONVOLUTIONAL NEURAL NETWORKS	3
	2.4 FASHION-MNIST DATASET	4
	2.5 RESEARCH GAP	5
3	**SYSTEM REQUIREMENTS**	**6**
	3.1 HARDWARE REQUIREMENTS	6
	3.2 SOFTWARE REQUIREMENTS	6
	3.3 LIBRARIES USED	7
4	**SYSTEM OVERVIEW**	**8**
	4.1 EXISTING SYSTEM	8
	4.1.1 DRAWBACKS OF EXISTING SYSTEM	8
	4.2 PROPOSED SYSTEM	8
	4.2.1 ADVANTAGES OF PROPOSED SYSTEM	9
5	**SYSTEM IMPLEMENTATION**	**10**
	5.1 SYSTEM ARCHITECTURE DIAGRAM	10
	5.2 SYSTEM FLOW	11
	5.3 LIST OF MODULES	12
	5.4 MODULE DESCRIPTION	13
6	**RESULTS AND DISCUSSION**	**17**
	6.1 IMPLEMENTATION RESULTS	17
	6.2 RESULTS DISCUSSION	18
	6.3 PERFORMANCE EVALUATION	18
	6.4 LIMITATIONS	19
	6.5 CONCLUSION	19
7	**APPENDIX**	**21**
	7.1 SAMPLE CODE	21
	7.2 OUTPUT SCREENSHOTS	24
	REFERENCES	**27**

CHAPTER 1

INTRODUCTION

1.1 INTRODUCTION

Image classification is a fundamental task in computer vision that involves assigning predefined categories to images. It is widely used in applications such as object recognition, medical image analysis, product categorization, and automated visual inspection.

Fashion item classification focuses on identifying different types of clothing and accessories from images. It can be useful in online shopping platforms, digital catalog management, clothing recommendation systems, and automated product organization.

Traditional image classification techniques often depend on manually designed features. However, deep learning enables neural networks to learn useful visual features directly from training images.

A Multilayer Perceptron (MLP) is a type of artificial neural network that processes input features through fully connected layers. When applied to images, the pixel values are typically flattened into a one-dimensional vector. Although this approach can perform image classification, it does not explicitly preserve the spatial relationships between neighboring pixels.

A Convolutional Neural Network (CNN) is designed to process image data. It uses convolutional layers to learn local patterns and pooling layers to reduce spatial dimensions. These features make CNNs suitable for recognizing visual patterns in images.

This project implements two models, a baseline MLP and a CNN, to classify fashion images using the Fashion-MNIST dataset. Both models are trained and evaluated under the same experimental conditions to compare their performance.

The implementation uses Python, TensorFlow, Keras, NumPy, Matplotlib, Seaborn, and Scikit-learn. The project generates evaluation metrics and visualizations to analyze the strengths and limitations of the two approaches.

1.2 OBJECTIVES

The main objectives of the project are:

- To develop a fashion image classification system using deep learning.
- To load and preprocess the Fashion-MNIST dataset.

- To implement a baseline Multilayer Perceptron neural network.
- To develop a Convolutional Neural Network for image classification.
- To train both models using training and validation datasets.
- To evaluate the models using accuracy, precision, recall, and F1-score.
- To generate training performance graphs and confusion matrices.
- To compare the classification performance of the MLP and CNN.
- To save the trained models for future predictions.

1.3 SCOPE OF THE PROJECT

The project focuses on classifying grayscale fashion images into ten predefined categories using deep learning.

The dataset contains images representing T-shirts, trousers, pullovers, dresses, coats, sandals, shirts, sneakers, bags, and ankle boots.

The project covers dataset loading, preprocessing, model development, training, validation, testing, performance evaluation, visualization, and model saving.

The baseline MLP uses flattened image pixels as input features, while the CNN processes the images using convolutional and pooling operations.

The system is designed as an academic mini project to demonstrate and compare two neural network architectures. It can be extended in the future by applying data augmentation, hyperparameter tuning, transfer learning, and more advanced CNN architectures.

The current implementation uses small grayscale images from the Fashion-MNIST dataset. Its performance on high-resolution images or real-world fashion photographs may differ.

CHAPTER 2

LITERATURE REVIEW

2.1 DEEP LEARNING FOR IMAGE CLASSIFICATION

Deep learning is a branch of machine learning that uses neural networks with multiple layers to learn patterns from data.

In image classification, deep learning models learn visual features from training images and use these features to predict the category of an unseen image.

The learned features can include edges, textures, shapes, and combinations of visual patterns. The effectiveness of a model depends on the dataset, network architecture, training procedure, and evaluation method.

Deep learning frameworks such as TensorFlow and Keras simplify the development, training, and evaluation of neural networks.

2.2 MULTILAYER PERCEPTRON FOR CLASSIFICATION

A Multilayer Perceptron is a feed-forward artificial neural network consisting of an input layer, one or more hidden layers, and an output layer.

For image classification, the input image is flattened into a one-dimensional array before being passed to the dense layers.

In this project, the baseline MLP consists of:

- An input layer accepting images of shape $28 \times 28 \times 1$.
- A flattening layer that converts the image into a one-dimensional vector.
- A dense layer containing 128 neurons with ReLU activation.
- An output layer containing ten neurons with softmax activation.

The softmax output represents the model's predicted scores across the ten fashion categories.

The MLP provides a baseline against which the CNN can be compared.

2.3 CONVOLUTIONAL NEURAL NETWORKS

A Convolutional Neural Network is a deep learning architecture commonly used for image classification.

CNNs apply convolutional filters to image data to learn local visual features. Pooling layers

reduce the spatial dimensions of feature maps, while dense layers use the extracted features to predict the output category.

The CNN implemented in this project consists of:

- An input layer accepting 28 × 28 grayscale images.
- A convolutional layer with 32 filters and ReLU activation.
- A max-pooling layer.
- A convolutional layer with 64 filters and ReLU activation.
- A second max-pooling layer.
- A flattening layer.
- A dense layer containing 128 neurons.
- A dropout layer with a rate of 0.4.
- An output layer containing ten neurons with softmax activation.

The dropout layer helps reduce overfitting by randomly deactivating a proportion of units during training.

The CNN is evaluated against the baseline MLP to determine which architecture performs better in the experiment.

2.4 FASHION-MNIST DATASET

Fashion-MNIST is a dataset used for image classification research and educational experiments. It contains 70,000 grayscale images of fashion items, each measuring 28 × 28 pixels.

The dataset is divided into 60,000 training images and 10,000 test images.

The ten categories are:

Class Label	Fashion Category
0	T-shirt/top
1	Trouser
2	Pullover
3	Dress
4	Coat
5	Sandal
6	Shirt
7	Sneaker
8	Bag
9	Ankle boot

Table 2.1: Fashion-MNIST Class Labels

The dataset is loaded automatically using

`tf.keras.datasets.fashion_mnist.load_data()`.

The original training set is further divided into training and validation subsets using an 80:20 split. This produces 48,000 training images and 12,000 validation images, while the original 10,000 test images remain separate.

2.5 RESEARCH GAP

Image classification can be implemented using different neural network architectures, but their performance may vary depending on how image features are represented and learned.

An MLP processes flattened image pixels, whereas a CNN learns spatial features through convolutional operations.

This project addresses the comparison of these two approaches using the same Fashion-MNIST dataset, training settings, validation data, and test data.

The project evaluates both models using multiple metrics and graphical visualizations rather than relying only on training accuracy.

The results provide a practical understanding of how neural network architecture influences fashion image classification.

CHAPTER 3

SYSTEM REQUIREMENTS

3.1 HARDWARE REQUIREMENTS

The following hardware is suitable for implementing the project:

- **Processor:** Intel or AMD processor.
- **RAM:** 4 GB minimum; 8 GB or more recommended.
- **Storage:** Sufficient free space for Python libraries, notebook files, and generated outputs.
- **GPU:** Optional; GPU acceleration may reduce training time.
- **Display:** Standard laptop or desktop display.
- **Internet Connection:** Required for the initial automatic download of the Fashion-MNIST dataset when it is not already available locally.

The experiment can be implemented using Google Colab or a local Python environment.

3.2 SOFTWARE REQUIREMENTS

Software	Purpose
Operating System	Windows, Linux, or macOS
Programming Language	Python
Development Environment	Google Colab or Jupyter Notebook
Deep Learning Framework	TensorFlow
Neural Network API	Keras
Numerical Processing	NumPy
Data Visualization	Matplotlib and Seaborn
Model Evaluation	Scikit-learn
Dataset	Fashion-MNIST
Version Control	Git and GitHub

Table 3.1: Software Requirements

3.3 LIBRARIES USED

Library	Purpose
NumPy	Numerical operations and array manipulation
Matplotlib	Plotting training curves and comparison graphs
Seaborn	Visualizing confusion matrices
TensorFlow	Deep learning model development and training
Keras	Building, compiling, training, and saving neural networks
Scikit-learn	Splitting data and calculating evaluation metrics

Table 3.2: Libraries Used

The libraries are imported at the beginning of the program.

The main deep learning operations are performed using TensorFlow and Keras, while Scikit-learn is used to calculate classification metrics and generate confusion matrices.

CHAPTER 4

SYSTEM OVERVIEW

4.1 EXISTING SYSTEM

Traditional image classification methods often rely on manually designed features and conventional machine learning algorithms.

For fashion image classification, an image may be converted into numerical features before being supplied to a classification algorithm. The effectiveness of this approach depends on the quality of the extracted features and the selected algorithm.

A basic neural network can also classify images by flattening their pixel values and processing them through fully connected layers.

In this project, the baseline MLP represents the conventional fully connected neural network approach. Its performance is compared with a CNN that learns spatial features directly from images.

4.1.1 DRAWBACKS OF EXISTING SYSTEM

The baseline MLP approach has the following limitations:

- Flattening images removes their explicit two-dimensional spatial structure.
- The model does not directly use convolutional filters to learn local image patterns.
- The number of parameters can increase when fully connected layers process high-dimensional images.
- Performance may be limited when the relationships between neighboring pixels are important.
- The model may confuse visually similar fashion categories.

These limitations motivate the implementation of a CNN for comparison.

4.2 PROPOSED SYSTEM

The proposed system implements and compares two deep learning models for fashion item image classification.

The first model is a baseline MLP consisting of a flattening layer, a dense hidden layer with 128 neurons, and a softmax output layer.

The second model is a CNN containing two convolutional layers, two max-pooling layers, a

dense layer, a dropout layer, and a softmax output layer.

The Fashion-MNIST dataset is loaded using TensorFlow/Keras. Image pixel values are normalized from the range 0–255 to 0–1. A channel dimension is added to the images to make them compatible with the CNN input format.

The training dataset is split into training and validation subsets. Both models are trained for ten epochs using a batch size of 128 and the Adam optimizer.

After training, both models are evaluated on the same test dataset. Accuracy, precision, recall, and F1-score are calculated. The program also generates training graphs, confusion matrices, and model comparison charts.

Finally, the trained models are saved in `.keras` format for future use.

4.2.1 ADVANTAGES OF PROPOSED SYSTEM

- Implements two neural network architectures for comparison.
- Uses the Fashion-MNIST dataset for a standardized experiment.
- Automatically downloads the dataset through Keras when required.
- Normalizes image pixel values for model training.
- Uses a separate validation dataset to monitor model performance.
- Evaluates models using multiple classification metrics.
- Generates graphs to analyze training and validation performance.
- Uses confusion matrices to identify classification errors.
- Displays predictions and confidence scores for a sample image.
- Saves trained models for future inference.

CHAPTER 5

SYSTEM IMPLEMENTATION

5.1 SYSTEM ARCHITECTURE DIAGRAM

The system consists of the following main components:

- Fashion-MNIST dataset.
- Data preprocessing module.
- Training and validation data split.
- Baseline MLP model.
- CNN model.
- Model training module.
- Model evaluation module.
- Visualization and comparison module.
- Sample prediction module.
- Model saving module.

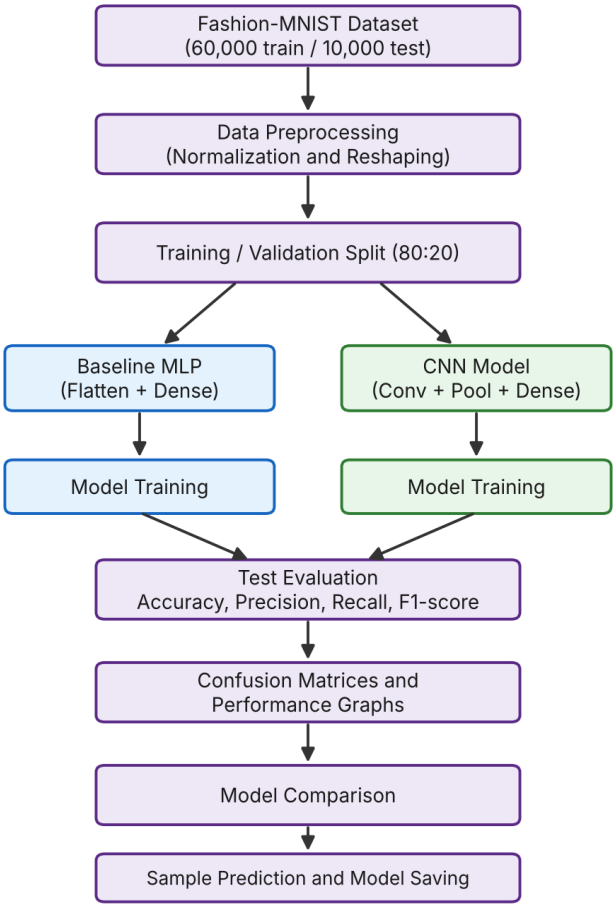


Fig. 5.1: System Architecture of Fashion Item Image Classification

5.2 SYSTEM FLOW

The system follows a sequence of operations to train and evaluate the two classification models.

1. Initialize the random seeds and training parameters.
2. Import the required Python libraries.
3. Load the Fashion-MNIST dataset.
4. Display sample fashion images.
5. Normalize the pixel values to the range 0–1.
6. Add a channel dimension to the images.
7. Split the original training data into training and validation sets.
8. Build the baseline MLP model.
9. Train the baseline MLP for ten epochs.
10. Build the CNN model.
11. Train the CNN for ten epochs.
12. Generate training and validation accuracy and loss graphs.
13. Evaluate both models on the test dataset.

14. Calculate accuracy, precision, recall, and F1-score.
15. Generate confusion matrices for both models.
16. Compare the evaluation metrics and parameter counts.
17. Predict the class of a sample test image.
18. Save both trained models and generated output images.

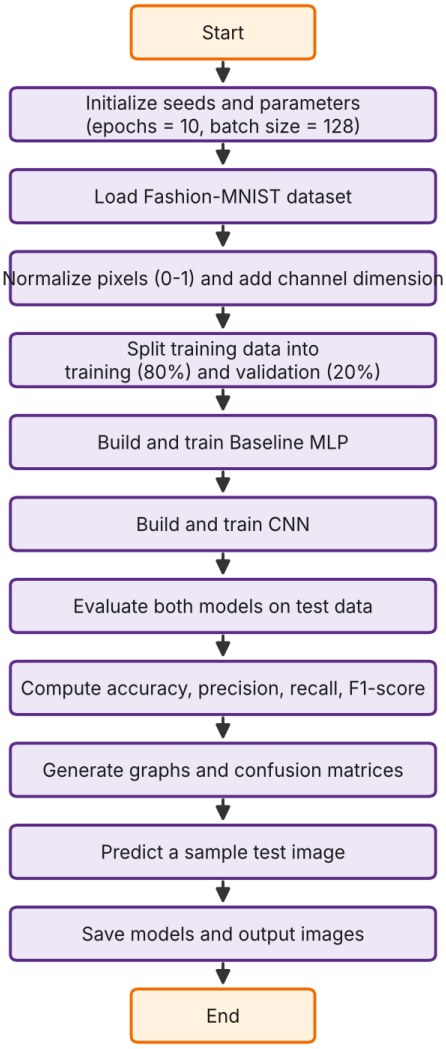


Fig. 5.2: System Flow of Fashion Item Image Classification

5.3 LIST OF MODULES

The project consists of the following modules:

- Initialization Module
- Dataset Loading Module
- Data Preprocessing Module
- Baseline MLP Development Module
- CNN Development Module

- Model Training Module
- Performance Evaluation Module
- Visualization Module
- Model Comparison Module
- Sample Prediction Module
- Model Saving Module

5.4 MODULE DESCRIPTION

5.4.1 Initialization Module

This module imports the required libraries and initializes the experimental parameters.

The random seed is set to 42 for NumPy and TensorFlow. The number of training epochs is set to 10, and the batch size is set to 128.

The ten fashion class names are also defined for use in visualizations and classification reports.

5.4.2 Dataset Loading Module

This module loads the Fashion-MNIST dataset using the Keras dataset API.

The dataset is divided into training and test sets. The training set contains 60,000 images, and the test set contains 10,000 images.

Each image is represented as a 28 × 28 grayscale array, and its label indicates the corresponding fashion category.

Sample images are displayed and saved as `sample_images.png`.

5.4.3 Data Preprocessing Module

This module prepares the image data for neural network training.

The images are converted to the `float32` data type and divided by 255 to normalize the pixel values to the range 0–1.

A channel dimension is added to represent the grayscale channel. The resulting image shape is 28 × 28 × 1.

The training dataset is then divided into training and validation subsets using `train_test_split()`. The validation set represents 20% of the original training data, and stratification helps preserve the distribution of class labels.

5.4.4 Baseline MLP Development Module

This module builds the baseline Multilayer Perceptron model.

The input image is passed through a flattening layer, which converts the 28 × 28 image into a one-dimensional vector.

A dense layer containing 128 neurons uses ReLU activation to learn patterns from the input. The output layer contains ten neurons with softmax activation to produce scores for the ten fashion categories.

The model uses the Adam optimizer, sparse categorical cross-entropy loss, and accuracy as the training metric.

5.4.5 CNN Development Module

This module builds the Convolutional Neural Network.

The architecture contains two convolutional layers with 32 and 64 filters, respectively. Each convolutional layer uses a 3 × 3 filter size, ReLU activation, and same padding.

Two max-pooling layers reduce the spatial dimensions of the feature maps. The resulting features are flattened and passed through a dense layer containing 128 neurons.

A dropout layer with a rate of 0.4 is used to help reduce overfitting. The final dense layer contains ten neurons with softmax activation.

The CNN uses the Adam optimizer and sparse categorical cross-entropy loss.

5.4.6 Model Training Module

This module trains both the baseline MLP and the CNN.

Each model is trained using the training dataset for ten epochs with a batch size of 128. The validation dataset is used to monitor performance during training.

The training histories store accuracy, validation accuracy, loss, and validation loss for each epoch.

These values are used to analyze the learning behavior of both models.

5.4.7 Performance Evaluation Module

This module evaluates the trained models using the test dataset.

The model generates predicted probabilities for each test image. The predicted class is obtained by selecting the index with the highest probability.

The following metrics are calculated:

- Accuracy
- Macro precision
- Macro recall
- Macro F1-score

A classification report is also generated to provide precision, recall, F1-score, and support for each fashion category.

Confusion matrices are calculated to identify correct classifications and misclassifications.

5.4.8 Visualization Module

This module generates graphs to help interpret the experimental results.

The training performance graph compares the training and validation accuracy and loss of the two models.

Confusion matrices visualize the number of correct and incorrect predictions for each category.

The model comparison graph displays the accuracy, precision, recall, and F1-score of both models.

The generated graphs are saved as PNG images for inclusion in the report.

5.4.9 Sample Prediction Module

This module demonstrates the classification of an individual test image.

The image is passed to both trained models. The predicted category and confidence score are obtained from each model.

The image is displayed alongside the actual category and the predictions made by the baseline MLP and CNN.

The output is saved as `sample_prediction.png`.

5.4.10 Model Saving Module

This module saves both trained models in Keras format.

The baseline MLP is saved as `baseline_model.keras`, and the CNN is saved as `fashion_cnn_model.keras`.

Saving the trained models allows them to be loaded for future predictions without repeating the entire training process.

CHAPTER 6

RESULTS AND DISCUSSION

6.1 IMPLEMENTATION RESULTS

The project implements two deep learning models for fashion item image classification using the Fashion-MNIST dataset. The baseline MLP and CNN are trained and evaluated using the same dataset split.

The program performs dataset loading, image preprocessing, model development, training, testing, and performance visualization.

The following table summarizes the expected execution outputs produced by the code.

S. No.	Component	Expected Output
1	Dataset Loading	60,000 training images and 10,000 test images
2	Data Preprocessing	Normalized images with pixel values between 0 and 1
3	Data Splitting	48,000 training images and 12,000 validation images
4	Baseline MLP	Trained neural network for ten fashion categories
5	CNN	Trained convolutional neural network for ten fashion categories
6	Model Evaluation	Accuracy, precision, recall, and F1-score
7	Training Visualization	Accuracy and loss graphs
8	Confusion Matrices	Classification results for both models
9	Model Comparison	Comparison of evaluation metrics
10	Sample Prediction	Actual class, predicted classes, and confidence scores
11	Model Saving	Two saved `.keras` model files

Table 6.1: Implementation Outputs

The program also generates five image files:

- `sample_images.png`
- `training_performance.png`
- `confusion_matrices.png`
- `model_comparison.png`
- `sample_prediction.png`

The trained models are saved as `baseline_model.keras` and `fashion_cnn_model.keras`.

6.2 RESULTS DISCUSSION

The project compares two neural network architectures with different approaches to image processing.

The baseline MLP converts each image into a one-dimensional vector before processing it through fully connected layers. This allows the model to learn relationships between pixel values but does not explicitly preserve the spatial arrangement of image features.

The CNN processes images using convolutional filters and pooling operations. These layers help the model learn local visual patterns and reduce the spatial dimensions of feature maps.

The training and validation accuracy graphs help assess the learning progress of each model. The corresponding loss curves show how prediction errors change over the training epochs.

The confusion matrices help identify which fashion categories are classified correctly and which categories are frequently confused.

The model comparison graph provides a direct comparison of accuracy, precision, recall, and F1-score.

CNNs are designed to exploit spatial information in images, so the CNN may perform better than the baseline MLP. However, the actual performance difference must be determined from the test results produced by the program.

6.3 PERFORMANCE EVALUATION

The performance of both models is evaluated using the following metrics.

Metric	Description
Accuracy	Measures the proportion of correctly classified test images.
Precision	Measures the proportion of predicted instances of a class that are correct.
Recall	Measures the proportion of actual instances of a class that are correctly identified.
F1-score	Combines precision and recall using their harmonic mean.
Confusion Matrix	Shows correct and incorrect predictions across the ten categories.

Table 6.2: Evaluation Metrics

The project calculates macro-averaged precision, recall, and F1-score. Macro averaging calculates each class's metric separately and then takes the unweighted mean across the ten classes.

Model Comparison Table

Evaluation Metric	Baseline MLP	CNN
Accuracy	_____	_____
Macro Precision	_____	_____
Macro Recall	_____	_____
Macro F1-score	_____	_____
Number of Parameters	_____	_____

Table 6.3: Performance Comparison of Baseline MLP and CNN

Interpretation

- **Accuracy:** Indicates the overall classification performance.
- **Precision:** Helps assess the reliability of predicted categories.
- **Recall:** Shows how effectively each model identifies images belonging to the actual categories.
- **F1-score:** Provides a combined measure of precision and recall.
- **Confusion Matrix:** Helps identify categories that are frequently misclassified.
- **Parameter Count:** Indicates the number of parameters in each model, which provides information about model size and complexity.

6.4 LIMITATIONS

The project has the following limitations:

- The models are trained on small grayscale images measuring 28 × 28 pixels.
- Performance on real-world fashion photographs may differ from performance on Fashion-MNIST images.
- Visually similar categories, such as shirts, pullovers, and coats, may be confused.
- The models are trained for a fixed number of ten epochs, without automatic early stopping.
- Hyperparameter tuning is not included in the current implementation.
- The CNN may require more computational resources than the baseline MLP.
- The experiment does not include transfer learning or advanced CNN architectures.
- The sample prediction demonstrates only one test image and does not independently establish overall model performance.

CONCLUSION

This project implements and compares a baseline Multilayer Perceptron and a Convolutional Neural Network for fashion image classification using the Fashion-MNIST dataset.

The dataset is preprocessed by normalizing pixel values and preparing the images for model training. Both models are trained using the same training and validation datasets and evaluated on the same test dataset.

The project calculates accuracy, precision, recall, and F1-score and generates training performance graphs, confusion matrices, and model comparison charts. These outputs provide insights into the classification performance of the two neural network architectures.

The experiment demonstrates the application of deep learning to computer vision and provides practical experience in dataset preprocessing, neural network implementation, model training, and performance evaluation.

The final comparison identifies which model performs better under the selected experimental settings. Future improvements may include hyperparameter tuning, data augmentation, advanced CNN architectures, and testing on additional image datasets.

CHAPTER 7

APPENDIX

PROJECT FILES

The following files can be included in the GitHub repository for the project.

File Name	Description
`fashion_classification.ipynb`	Jupyter Notebook containing the complete Python implementation
`README.md`	Project overview, objectives, implementation steps, results, and conclusion
`requirements.txt`	List of required Python libraries
`sample_images.png`	Sample images from the Fashion-MNIST dataset
`training_performance.png`	Training and validation accuracy and loss graphs
`confusion_matrices.png`	Confusion matrices for both models
`model_comparison.png`	Comparison of classification metrics
`sample_prediction.png`	Sample image prediction output
`baseline_model.keras`	Trained baseline MLP model
`fashion_cnn_model.keras`	Trained CNN model
`report.pdf`	Final project documentation

Table 7.1: Project Files

7.1 SAMPLE CODE

A. Loading the Fashion-MNIST Dataset

The following code loads the dataset and displays its dimensions.

```
import tensorflow as tf

(x_train, y_train), (x_test, y_test) = (
    tf.keras.datasets.fashion_mnist.load_data()
)

print("Training images:", x_train.shape)
print("Training labels:", y_train.shape)
print("Test images:", x_test.shape)
print("Test labels:", y_test.shape)
```

B. Data Preprocessing

The images are normalized, and a channel dimension is added.

```
import numpy as np

x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

x_train = x_train[..., np.newaxis]
x_test = x_test[..., np.newaxis]

print("Training shape:", x_train.shape)
print("Test shape:", x_test.shape)
```

C. Building the Baseline MLP

```
from tensorflow.keras import layers, models

baseline_model = models.Sequential([
    layers.Input(shape=(28, 28, 1)),
    layers.Flatten(),
    layers.Dense(128, activation="relu"),
    layers.Dense(10, activation="softmax")
])

baseline_model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

baseline_model.summary()
```

D. Building the CNN

```
cnn_model = models.Sequential([
    layers.Input(shape=(28, 28, 1)),

    layers.Conv2D(
        32, (3, 3),
        activation="relu",
        padding="same"
    ),
    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(
        64, (3, 3),
        activation="relu",
        padding="same"
    ),
    layers.MaxPooling2D((2, 2)),

    layers.Flatten(),
    layers.Dense(128, activation="relu"),
    layers.Dropout(0.4),
    layers.Dense(10, activation="softmax")
])

cnn_model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

cnn_model.summary()
```

E. Model Evaluation

```
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score
)

predictions = cnn_model.predict(x_test, verbose=0)
y_pred = np.argmax(predictions, axis=1)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(
    y_test, y_pred, average="macro"
))
print("Recall:", recall_score(
    y_test, y_pred, average="macro"
))
print("F1-score:", f1_score(
    y_test, y_pred, average="macro"
))
```

Note: The model must be trained before running the evaluation code. The complete experiment, including the training and comparison of both models, is provided in the main Jupyter Notebook.

7.2 OUTPUT SCREENSHOTS

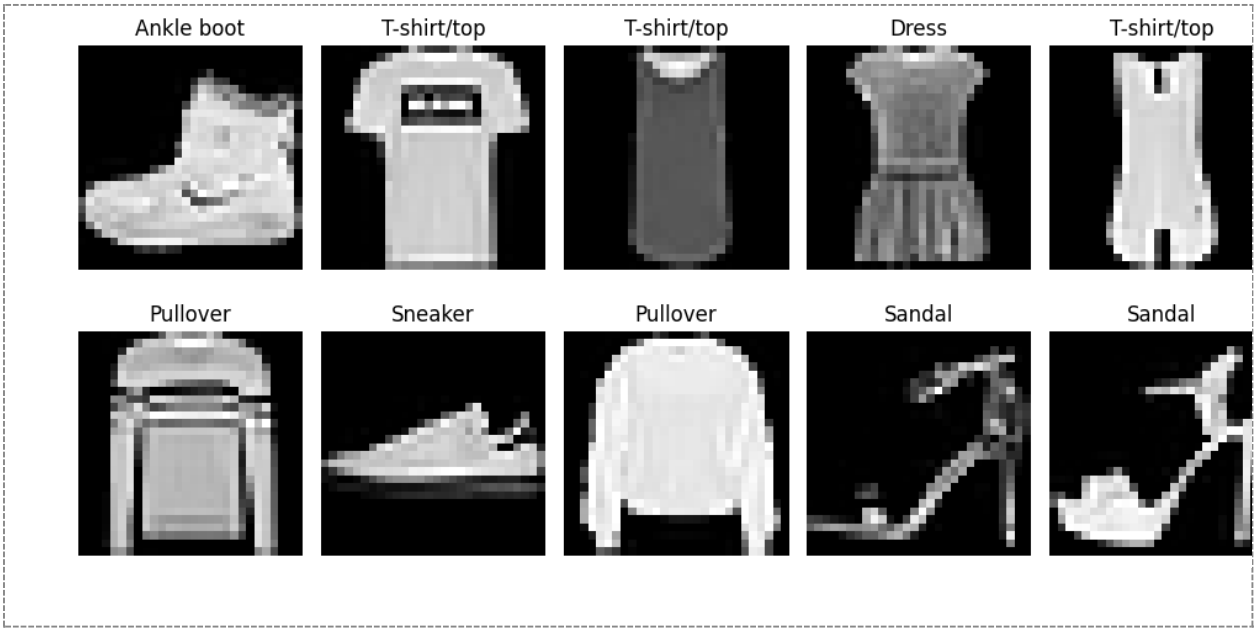


Fig. 7.1: Sample Fashion-MNIST Images

This figure displays sample grayscale fashion images and their corresponding category labels.

Model: "baseline_mlp"

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 784)	0
dense (Dense)	(None, 128)	100,480
dense_1 (Dense)	(None, 10)	1,290

Total params: 101,770 (397.54 KB)
Trainable params: 101,770 (397.54 KB)
Non-trainable params: 0 (0.00 B)

Fig. 7.2: Training and Validation Performance

This figure compares training and validation accuracy and loss for the baseline MLP and CNN over ten epochs.

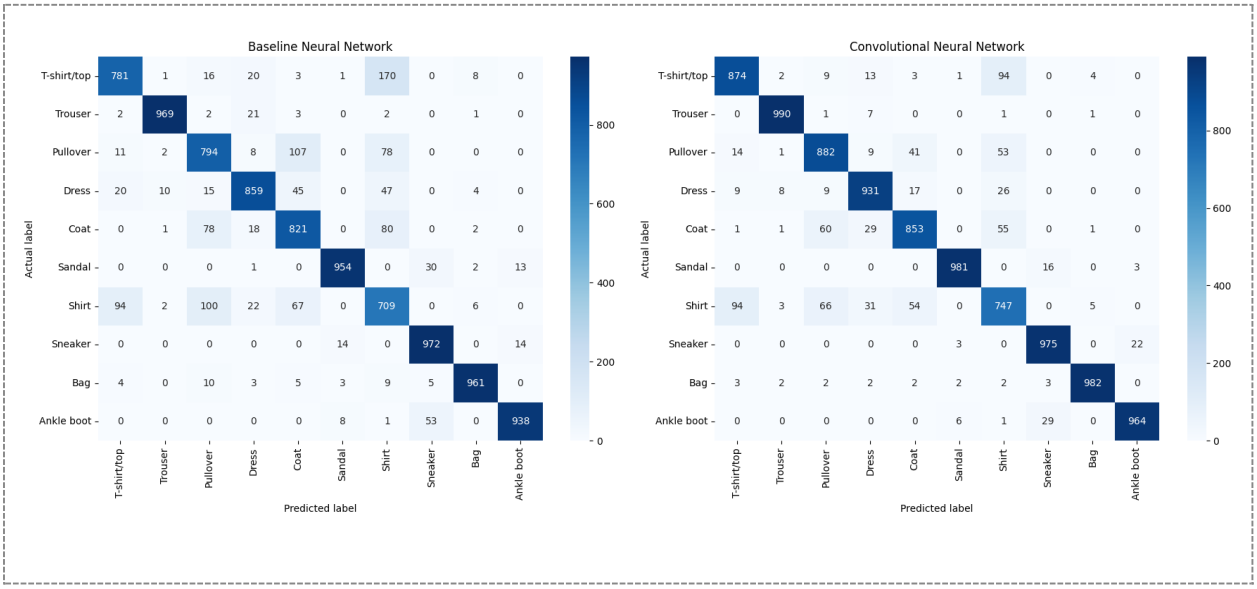


Fig. 7.3: Confusion Matrices

This figure shows the classification performance of both models across the ten fashion categories.

Metric	*Baseline*	*CNN*
Accuracy	*0.8758*	*0.9179*
Precision	*0.8792*	*0.9177*
Recall	*0.8758*	*0.9179*
F1-score	*0.8769*	*0.9177*

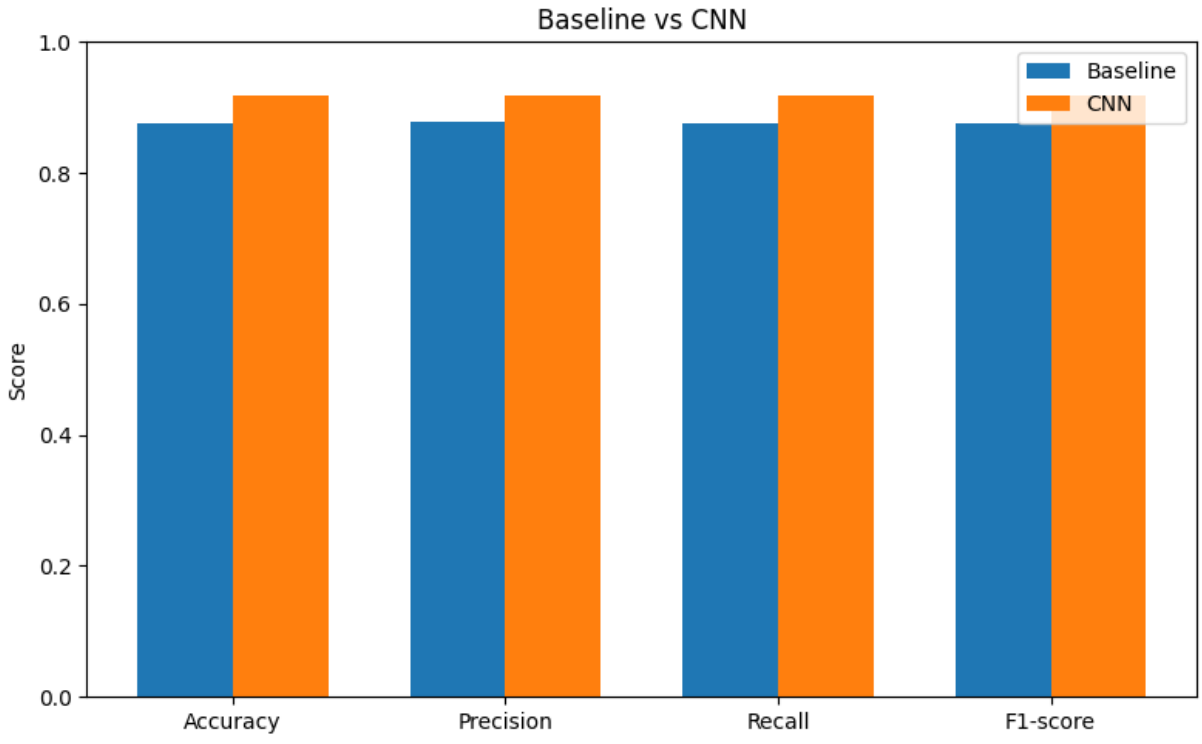
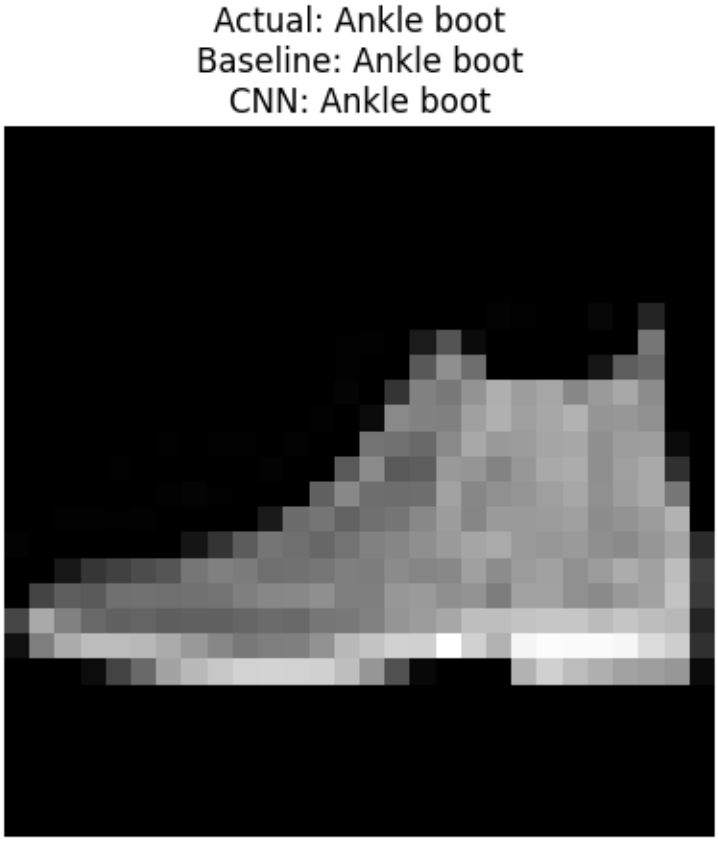


Fig. 7.4: Baseline MLP vs CNN Comparison

This figure compares accuracy, precision, recall, and F1-score for both models.



Actual class: Ankle boot
*<function print(*args, sep=' ', end='\n', file=None, flush=False)>*

Fig. 7.5: Sample Image Prediction

This figure displays a test image, its actual category, the predictions from both models, and their confidence scores.

REFERENCES

[1] TensorFlow. *TensorFlow Documentation.* Available at: https://www.tensorflow.org/

[2] Keras. *Keras Documentation.* Available at: https://keras.io/

[3] Fashion-MNIST. *Fashion-MNIST Dataset.* Available at: https://github.com/zalandoresearch/fashion-mnist

[4] Scikit-learn. *Scikit-learn Documentation.* Available at: https://scikit-learn.org/stable/

[5] NumPy. *NumPy Documentation.* Available at: https://numpy.org/doc/

[6] Matplotlib. *Matplotlib Documentation.* Available at: https://matplotlib.org/stable/contents.html

[7] Seaborn. *Seaborn Documentation.* Available at: https://seaborn.pydata.org/